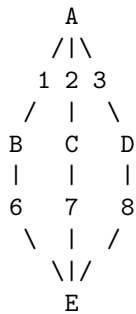


Assignment 5

1. [4 marks] Give an example of a connected, weighted, undirected graph and a start vertex such that neither BFS-tree nor DFS-tree is an MST, regardless of how the adjacency lists are ordered. Prove that your graph and start vertex satisfy the above requirements.

Graph and Start Vertex



Edges: $(A,B)=1$, $(A,C)=2$, $(A,D)=3$, $(B,E)=6$, $(C,E)=7$, $(D,E)=8$ **Start vertex:** A

Proof

MST: Edges $\{(A,B), (A,C), (A,D), (B,E)\}$ with weight = $1+2+3+6 = 12$

BFS from A:

- discover B, C, D at level 1 then add all three edges from A
- discover E at level 2 through any of $\{B,C,D\}$ to appear first in a adjacency list
- if C explored before B: add $(C,E)=7$ which gives a total weight of 13 $\neq 12$
- BFS is not MST

DFS from A:

- explore along one branch before backtracking
- if ordered $A \rightarrow B \rightarrow E$, then backtrack: add $(D,E)=8$, $(A,C)=2$, $(A,B)=1$
- total weight is 14 $\neq 12$
- DFS is not MST

With an appropriate adjacency list order, neither BFS or DFS produces the MST from vertex A.

2. [6 marks] Prove or disprove each of the following statements, where in each case $G = (V, E)$ is a connected undirected weighted graph with n vertices and m edges.
1. If all weights of edges in G are distinct then the lightest edge e is part of every minimum spanning tree of G .
 2. If all weights of edges in G are distinct then the heaviest edge e is not part of any minimum spanning tree of G .
 3. If e is a maximal weight edge of a cycle of G , then there is a minimum spanning tree of G that does not include e .

part (a): Lightest edge in every MST

Statement: If all weights are distinct, the lightest edge e is in every MST

Answer: true

Proof: Let $e=(u,v)$ be the minimum weight edge. Consider cut $(S, V \setminus S)$ with $u \in S, v \notin S$. By the property, in some MST the minimum edge is crossing any cut. Since e is a global minimum, every MST has e . Suppose MST T excludes e . Then T has edge e' crossing this cut with $w(e') > w(e)$. e gives a lighter tree by replacing e' , which contradicts T being a MST.

part (b): Heaviest edge is no MST

Statement: If all weights are distinct, the heaviest edge e is not in any MST

Answer: false

Counterexample:

B---C---D
1 5

Only edges: $(B,C)=1, (C,D)=5$. The MST include both edges. $(C,D)=5$ is essential for connectivity because it is the heaviest edge.

part (c): Maximum weight edge in cycle

Statement: If e is a maximum weight of a cycle, there exists an MST without e

Answer: true

Proof: By Cycle Property, the maximum weight edge in any cycle with distinct weights is not in any MST.

Direct argument: If $e=(u,v)$ is in spanning tree T and is the maximum of cycle C , removing e creates components T_1, T_2 . Since C exists, there is an alternative edge e' connecting T_1 and T_2 with $w(e') < w(e)$. Then $T' = T - \{e\} + \{e'\}$ is a lighter spanning tree, contradicting T being a MST. Therefore, no MST contains e .

3. [7 marks] Let G be a directed acyclic graph. You would like to know if graph G contains a directed path that goes through every vertex exactly once. Give an algorithm that tests this property. Provide pseudo-code along with justification of the correctness and analyze running time complexity of your algorithm. Your algorithm must have a running time in $O(|V| + |E|)$.

Main idea

A DAG has a Hamiltonian path if its topological ordering has all consecutive vertices connected by edge.

Pseudocode

```
HasHamiltonianPath(G):
    // G = (V,E) is a DAG
    //  $O(|V| + |E|)$ 
    topo = TopologicalSort(G)

    for i = 0 to |topo| - 2:
        u = topo[i]
        v = topo[i+1]
        if (u,v) ! E:
            return False

    return True

TopologicalSort(G):
    visited = null
    stack = []

    DFS-Visit(v):
        visited.add(v)
        for each neighbor w of v:
            if w ! visited:
                DFS-Visit(w)
        stack.push(v)

    for each neighbor v of v:
        if v ! visited:
            DFS-Visit(v)

    return reverse(stack)
```

Correctness

Key Lemma: Since edges only go “forward” any Hamiltonian path is a DAG must follow a topological order. ($\rightarrow$) If algorithm returns True, consecutive edges from path visit all vertices at once. ($\leftarrow$) If Hamiltonian path exists, it respects topological order. The sort produces a valid ordering that will detect consecutive edges.

Time Complexity

- TopologicalSort: $O(|V| + |E|)$
- Check consecutive edges: $O(|V|)$
- **Total:** $O(|V| + |E|)$

4. [4 marks] Consider a BFS and DFS of a connected undirected graph G that start from the same vertex s . If both sets of tree-edges (BFS-tree and DFS-tree) are equal to each other, then G is a tree. Prove or disprove.

Statement: If BFS and DFS from s have equal tree-edge sets, then G is a tree.

Answer: True

Proof

Both BFS and DFS produce spanning trees with $n-1$ edges. Given $T_BFS = T_DFS$.

Key Observations: If G has a cycle, it contains more than $n-1$ edges. These extra edges would be non-tree edges, classified differently by BFS vs DFS. Since $T_BFS = T_DFS$, both algorithms classify every edge identically. This only happens when:

- G has exactly $n-1$ edges
- all edges are tree edges
- no cycle exists

Therefore G is a tree since $|E| = n-1$ and G is connected.

-
5. [9 marks] A *vertex-and-edge-weighted graph* is an undirected graph $G = (V, E)$ where each vertex $v \in V$ has a *cost* $c(v)$ and every edge $e \in E$ has a *weight* $w(e)$. The *length* of a path in G is the sum of the weights of the edges in the path *and* the cost of the vertices in the path. In the Single-Source Shortest Path (SSSP) problem on vertex-and-edge-weighted graphs, we are given G and a source vertex s and we want to determine the minimum length of a path from s to v for every $v \in V$. (Note that the minimum length of a path from s to s is $c(s)$.)

Solve the SSSP problem on vertex-and-edge-weighted graphs in the special case where all the weights and the costs are positive integers. Provide pseudo-code along with justification of the correctness and analyze running time complexity of your algorithm.

Main Idea

Modify the Dijkstra algorithm to include vertex costs when entering each vertex: $\text{dist}[s] = c(s)$, and relaxation adds both weight and destination vertex cost.

Pseudocode

```
VertexEdgeDijkstra(G, s, c, w):
    \ c[v] = vertex costs, w[e] = edge weights
    for each v in V:
        dist[v] = inf
    dist[s] = c[s]

    PQ = MinHeap with (C[s], s)
    visited = null
    while PQ is not empty:
        (d,u) = PQ.extractMin()

        if u in visited:
            continue
        visited.add(u)

        for each edge (u,v) in E:
            if v not in visited:
                newDist = dist[u] + w[(u,v)] + c[v]
                if newDist < dist[v]:
                    dist[v] = newDist
                    PQ.insert(newDist, v)

    return dist
```

Correctness

The path length from s to v = sum of all vertex costs on path + sum of all edge weights. The algorithm maintains: $\text{dist}[u]$ = shortest path length from s to u for all processed vertices.

Optimal substructure holds: Shortest path to v through u uses shortest path to u , plus edge (u,v) , plus vertex cost $c[v]$. Since all cost and weights are positive and greedy selection extracting the minimum from PQ is safe. Standard Dijkstra correctness argument applies.

Time Complexity

- With Binary heap: $O((|V| + |E|) \log |V|)$
 - With Fibonacci heap: $O(|V| \log |V| + |E|)$
-

6. [10 marks] Given a directed graph with positive edge lengths and a two distinct vertices x, y in the graph, the “all-pairs x, y -exclusively constrained shortest path problem” is the problem of computing for each pair of vertices i and j the shortest path from i to j that goes through the vertex x or through vertex y **but not through both of them**. If no such path exists, the answer is ∞ .
1. [3 marks] Describe in words an algorithm that takes a graph $G = (V, E, W)$, where W_{ij} represents length of edges, and vertices x, y as input parameters and computes values $L(i, j)$ that represent the length of x, y -exclusively constrained shortest path from i to j for all $1 \leq i, j \leq |V|$, $i \notin \{x, y\}$, $j \notin \{x, y\}$. **Make sure you clearly describe all transformations of the input graph required for your algorithm**
 2. [5 marks] Provide clear pseudocode solution.
 3. [2 marks] Justify correctness of your algorithm and provide the running time analysis. Your algorithm should have a running time in $O(|V|^2)$.

part (a): Main Idea

For each pair (i, j) , compute:

1. shortest path $i \rightarrow x \rightarrow j$ that avoids y entirely
2. shortest path $i \rightarrow y \rightarrow j$ that avoids x entirely
3. take minimum

Implementation: Run 4 Dijkstra algorithms:

- from x on G with y removed $\rightarrow$ `dist_from_x_no_y`
- to x on reversed G with y removed $\rightarrow$ `dist_from_x_no_y`
- from y on G with x removed $\rightarrow$ `dist_from_y_no_x`
- to y on reversed G with x removed $\rightarrow$ `dist_from_y_no_x`

Then: $L[i, j] = \min(\text{dist_from_x_no_y}[i] + \text{dist_from_x_no_y}[j], \text{dist_from_y_no_x}[i] + \text{dist_from_y_no_x}[j])$

part (b): Pseudocode

```
XY_EXclusiveShortestPaths(G, x, y):
    // G = (V, E, W) directed graph with positive weights
    n = |V|
    L = nxn matrix initialized to inf

    // Create modified graphs
    G_on_x = RemoveVertex(G, x)
    G_on_y = RemoveVertex(G, y)
    G_rev_on_x = RemoveVertex(G_no_x)
    G_rev_on_y = RemoveVertex(G_no_y)

    // Compute shortest paths for all pairs
    for each i in V \ {x,y}:
        for each j in V \ {x,y}:
            path_via_x = dist_to_x_no_y[i] + dist_to_x_no_y[j]
            path_via_y = dist_to_y_no_x[i] + dist_to_y_no_x[j]
            L[i,j] = min(path_via_x, path_via_y)

    return L

Dijkstrea(G, s):
    // Array-based implamentation for  $O(|V|^2)$ 
    for each v in V:
        dist[v] = inf
    dist[s] = 0
    visited = null

    repeat |V| times:
        u = vertex in V \ visited with minimum dist[u]
        if u = null:
            break
        visited.add(u)

        for each edge (u,v) in E:
            if dist[u] + w[(u,v)] < dist[v]
                dist[v] = dist[u] + w[(u,v)]

    return dist
```



```

RemoveVertex(G, v):
    // Returns graph with all edges reversed
    G' = empty graph
    for each v in V:
        G'.addVertex(v)
    for each edge (u,v) in E:
        G'.addEdge(v, u, weight)
    return G'

```

part (c): Correctness and Time Complexity

Correctness

Path through x only: Computing on $G_{\text{no_y}}$ ensures the path $i \rightarrow x \rightarrow j$ never use y. Sum $\text{dist_to_x_no_y}[i] + \text{dist_to_x_no_y}[j]$ gives shortest such path. **Path through y only:** Similarly, computing on $G_{\text{no_x}}$ ensures path $i \rightarrow y \rightarrow j$ never uses x. **Exculsive constraint satisfied:** Since graphs have one vertex removed, paths cannot use both x and y. Taking minimum gives shortest path using one of $\{x,y\}$.

Time Complexity: $O(|V|^2)$

- graph modifications: $O(|V| + |E|)$
- **4 dijkstra algorithm calls with array implementation:** $4 \times O(|V|^2) = O(|V|^2)$
- computing L matrix: $O(|V|^2)$ Array-based dijkstra algorithm guarantees $O(|V|^2)$ per call regardless of edge density.

Total: $O(|V|^2)$